Python Interview Questions

Table of Contents

Variables & Data Types 1
Operators 4
Conditional Statements 9
Loops 14
Python Data Structures 20
Functions 26
Exception Handling 31
File Handling 36
Python Object-Oriented Programming 42

Variables & Data Types

1. What is a variable in Python? Explain with an example.

Answer:
A variable in Python is a reserved memory location used to store values. Unlike many languages, Python does not require explicit declaration of data types; the type is assigned dynamically based on the value. Example:

```
x = 10
name = "John"
```

Here, `x` automatically becomes an integer and `name` becomes a string.

2. How does Python handle variable declaration?

Answer:
Python uses *dynamic typing*, meaning you do not declare the variable type explicitly. When a value is assigned, Python interprets the data type automatically. Example:

```
a = 5       # integer
a = "hello" # now string
```

This flexibility makes Python easy to use but requires careful handling to avoid type-related errors.

3. What are the basic data types in Python?

Answer:
Python's core built-in data types include:

- **int** – integers
- **float** – decimal numbers
- **str** – strings
- **bool** – True/False values
- **list** – ordered, mutable collection
- **tuple** – ordered, immutable collection
- **dict** – key-value pairs
- **set** – unordered collection of unique items
  Each type is an object in Python and supports various methods.

4. What is the difference between mutable and immutable data types?

Answer:

- **Mutable types** can be changed after creation. Examples: `list`, `dict`, `set`.

- **Immutable types** cannot be altered after creation. Examples: `int`, `float`, `str`, `tuple`. For instance:

```
x = "hello"
x[0] = "H"   # Error → string is immutable
```

But lists can change:

```
L = [1, 2, 3]
L[0] = 10    # Works
```

Immutability improves memory efficiency and reliability in multi-threading.

5. Explain type casting in Python with an example.

Answer:
Type casting converts one data type into another using built-in functions like `int()`, `float()`, `str()` etc.
Example:

```
x = "100"
num = int(x)  # string → integer
```

Casting is helpful when reading input from users because `input()` always returns string type.

6. What is the difference between = and == in Python?

Answer:

- = is the **assignment operator**, used to assign values to variables.
- == is the **comparison operator**, used to check if two values are equal.
  Example:

```
x = 10      # assignment
print(x == 10)  # comparison → True
```

7. How does Python store variables internally?

Answer:
In Python, every variable is a reference (pointer) to an object in memory. Variables do not store the actual value— they store the *reference* to the object.
Example:

```
a = 5
b = a
```

Both `a` and `b` point to the same integer object `5` in memory. Python uses an internal mechanism called *reference counting* to manage memory.

8. What is `type()` and `id()` in Python?

Answer:

- `type()` returns the data type of a variable.
- `id()` returns the unique memory address of the object.
  Example:

```
x = 20
print(type(x))  # <class 'int'>
print(id(x))    # unique memory location
```

These functions are useful for debugging and understanding object behavior.

9. What are numeric data types in Python?

Answer:
Python has three numeric types:

1. **int** – Whole numbers (unlimited length in Python 3).
2. **float** – Decimal numbers (double precision).
3. **complex** – Numbers with real and imaginary parts, written as `a + bj`.
   Example:

```
z = 3 + 4j
```

These types support mathematical operations and type conversions.

10. What is the difference between a list and a tuple in terms of data types?

Answer:

- **List** is **mutable**, meaning elements can be changed.
- **Tuple** is **immutable**; once created, its elements cannot be modified.
  Example:

```
L = [1, 2, 3]    # mutable
T = (1, 2, 3)    # immutable
```

Because tuples are immutable, they are faster and used for fixed data.

11. What is a dictionary in Python?

Answer:
A dictionary is a built-in data type that stores data in **key-value** pairs. Keys must be unique and immutable, while values can be any type. Example:

```
student = {"name": "Rohan", "age": 19}
```

Dictionaries are highly optimized for fast lookups.

12. What is a set and where is it used?

Answer:
A set is an **unordered collection of unique elements**. It automatically removes duplicates and is useful for mathematical operations like union, intersection, and difference. Example:

```
s = {1, 2, 2, 3}   # becomes {1, 2, 3}
```

13. Explain string data type characteristics.

Answer:
Strings are sequences of characters enclosed in quotes. They are **immutable**, support indexing, slicing, and many built-in methods. Example:

```
text = "Python"
text[0]   # 'P'
text.upper()  # 'PYTHON'
```

14. What is the difference between `None`, `False`, and `0` in Python?

Answer:

- `None` represents *absence of value*.
- `False` is a boolean value.
- `0` is a numeric value (int/float).
  Even though all three are considered "Falsy" in condition checks, they are completely different types.

15. Explain dynamic vs static typing with respect to Python.

Answer:

- **Static typing** (like Java, C++) requires declaring variable types at compile-time.
- **Dynamic typing** (Python) assigns types during runtime, based on the value.
  Example:

```
x = 10
x = "text"   # allowed in Python
```

Dynamic typing adds flexibility but can sometimes introduce runtime errors.

Operators

1. What are operators in Python? Explain with examples.

Answer:
Operators in Python are special symbols or keywords used to perform operations on variables and values. Python supports arithmetic, comparison, logical, assignment, membership, identity, and bitwise operators. Example:

```
a = 10 + 20     # '+' is an arithmetic operator
x = (5 > 3)     # '>' is a comparison operator
```

Operators help perform calculations, comparisons, and logical decisions in programs.

2. Explain arithmetic operators with examples.

Answer:
Arithmetic operators perform mathematical operations:

- `+` addition
- `-` subtraction
- `*` multiplication
- `/` division
- `//` floor division
- `%` modulus
- `**` exponent
  Example:

```
10 / 3   # 3.3333
10 // 3  # 3
2 ** 3   # 8
```

These are widely used for numerical calculations.

3. What are comparison (relational) operators? Give examples.

Answer:
Comparison operators compare two values and return `True` or `False`.
They include: `==`, `!=`, `>`, `<`, `>=`, `<=`.
Example:

```
5 == 5   # True
10 != 7  # True
4 > 9    # False
```

They are essential in decision-making statements like `if` conditions.

4. What are logical operators in Python?

Answer:
Logical operators combine conditional statements and return boolean results:

- `and` → True if both conditions are True
- `or` → True if at least one condition is True
- `not` → reverses boolean value
  Example:

```
x = (5 > 3 and 7 > 2)  # True
y = not (4 > 10)       # True
```

Used extensively in `if` conditions.

5. Explain assignment operators with examples.

Answer:
Assignment operators assign values to variables, often with shorthand expressions.
Examples:

- `=` simple assignment
- `+=` add & assign
- `-=` subtract & assign
- `*=` multiply & assign
- `/=` divide & assign
  Example:

```
x = 10
x += 5   # x = x + 5 → 15
```

They make code concise and efficient.

6. What are bitwise operators? Why are they used?

Answer:
Bitwise operators operate on the binary (bit) level of numbers. They are mainly used in low-level programming, networking, device drivers, and compression algorithms. Operators:

- `&` AND
- `|` OR
- `^` XOR
- `~` NOT
- `<<` left shift
- `>>` right shift
  Example:

```
5 & 3   # 1   (0101 & 0011 = 0001)
```

7. What are membership operators in Python?

Answer:
Membership operators check whether a value exists in a sequence (string, list, tuple, dict).

- `in` → returns True if value is present
- `not in` → returns True if value is not present
  Example:

```
'a' in "apple"     # True
10 not in [1,2,3]  # True
```

Useful for string and list processing.

8. What are identity operators? How are they different from equality operators?

Answer:
Identity operators compare memory locations, not values:

- `is`
- `is not`
  Example:

```
a = [1,2,3]
b = [1,2,3]
a == b    # True (values same)
a is b    # False (different objects in memory)
```

Equality (`==`) checks *value*, identity (`is`) checks *object identity*.

9. What is operator precedence? Give an example.

Answer:
Operator precedence determines the order in which operations are executed.
For example, multiplication has higher precedence than addition.

```
x = 10 + 2 * 3   # 10 + 6 = 16
```

Parentheses can be used to change the natural precedence.

10. What is operator associativity?

Answer:
Associativity decides the direction of evaluation when operators have the same precedence. Most operators are **left-associative**, meaning evaluated from left → right. Example:

```
10 - 2 - 1   # (10 - 2) - 1 = 7
```

Exponentiation `**` is right-associative:

```
2 ** 3 ** 2   # 2 ** (3 ** 2) = 2 ** 9 = 512
```

11. What are augmented assignment operators?

Answer:
Augmented assignment operators combine arithmetic operations with assignment in a single step.
Examples: `+=`, `-=`, `*=`, `/=`, `//=`, `%=`

```
x = 5
x *= 3   # x = x * 3 → 15
```

They improve code readability and reduce redundancy.

12. What is the difference between `/` and `//` operators?

Answer:

- `/` performs **true division**, returning floating-point results.
- `//` performs **floor division**, returning the largest integer less than or equal to the result.

```
10 / 3   # 3.3333
10 // 3  # 3
```

Used based on whether decimal precision is required.

13. How does Python's `not` operator work on non-boolean values?

Answer:
Python treats values as truthy or falsy:

- Falsy values: `0`, `None`, `""`, `[]`, `{}`, `False`
  Example:

```
not 0      # True
not "abc"  # False
```

`not` first converts the value to boolean, then negates it.

14. Explain short-circuit behavior in logical operators.

Answer:
Python's `and` and `or` operators stop evaluating as soon as the result is determined:

- `and` stops when it finds `False`
- `or` stops when it finds `True`
  Example:

```
x = 0 and print("Hello")   # print() doesn't execute
```

This improves performance and prevents unnecessary computation.

15. Can operators be overloaded in Python? Give example.

Answer:
Yes, Python supports operator overloading using special methods (dunder methods).
Example:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
```

Now + works with custom objects:

```
p1 + p2
```

This makes classes more intuitive and readable.

Conditional Statements

1. What are conditional statements in Python? Why are they used?

Answer:
Conditional statements allow a program to make decisions based on conditions. They help control the flow of execution by running different code blocks depending on whether a condition is True or False. Python uses `if`, `elif`, and `else` keywords to implement decision-making.
Example:

```
if age >= 18:
    print("Eligible to vote")
```

Without conditional statements, programs would always run sequentially, without logic-based branching.

2. Explain the syntax of the `if` statement with an example.

Answer:
The `if` statement checks a condition and executes a block of code only if the condition is True.
Syntax:

```
if condition:
    statement(s)
```

Example:

```
if 5 > 3:
    print("Yes")
```

The colon `:` indicates the start of a block, and indentation defines the block scope.

3. What is the purpose of `elif`? How is it different from `else`?

Answer:
`elif` (short for "else if") is used to check multiple conditions sequentially. If the `if` condition is False, Python checks the `elif` condition. Example:

```
if score > 90:
    grade = 'A'
elif score > 75:
    grade = 'B'
```

`else`, on the other hand, has no condition and only executes when all previous conditions fail.

4. How does the `else` statement work? Give an example.

Answer:
The `else` block executes only when all preceding `if` and `elif` conditions are False.
Example:

```
if x > 0:
    print("Positive")
else:
    print("Zero or Negative")
```

It acts as the default or fallback case in decision-making.

5. Can we use multiple `elif` statements in Python?

Answer:
Yes, you can use any number of `elif` statements to check multiple conditions.
Example:

```
if marks >= 90:
    grade = 'A'
elif marks >= 80:
    grade = 'B'
elif marks >= 70:
    grade = 'C'
else:
    grade = 'D'
```

Python evaluates conditions from top to bottom and stops at the first True condition.

6. What are nested `if` statements? Explain with an example.

Answer:
A nested `if` is an `if` statement inside another `if` block. It is used when decisions depend on more than one condition. Example:

```
if age > 18:
    if country == "India":
        print("Eligible to vote in India")
```

Although useful, excessive nesting can make code harder to read.

7. Explain the concept of "truthy" and "falsy" values in conditional statements.

Answer:
Python treats values as True or False automatically:
Falsy values: `0`, `""`, `[]`, `{}`, `None`, `False`, `()`
Everything else is considered **truthy**.
Example:

```
if []:
    print("True")
```

```
else:
    print("False")  # executes
```

This behavior simplifies conditions when checking empty containers or None.

8. What is the difference between `if condition:` and `if condition == True:`?

Answer:
`if condition:` is the correct and more pythonic approach because Python automatically evaluates the condition. Example:

```
is_raining = True
if is_raining:   # correct
```

Using `== True` is redundant and less efficient.

9. Can you write an `if` statement in a single line in Python?

Answer:
Yes, Python supports single-line conditional statements.
Example:

```
print("Yes") if x > 10 else print("No")
```

This is called a **ternary conditional expression** and is useful for simple conditions but not recommended for complex logic.

10. What is the ternary (conditional) operator in Python? Explain with example.

Answer:
Python's ternary operator allows compact conditional assignment:
Syntax:

```
value_if_true if condition else value_if_false
```

Example:

```
status = "Adult" if age >= 18 else "Minor"
```

It improves readability when the logic is simple.

11. How does Python evaluate chained conditional expressions?

Answer:
Python allows chained comparisons such as:

```
if 10 < x < 20:
```

This means:

```
10 < x and x < 20
```

It is cleaner, faster, and evaluated left to right.

12. Can we use logical operators inside conditional statements?

Answer:
Yes, logical operators (`and`, `or`, `not`) are frequently used inside `if` conditions to combine multiple checks.
Example:

```
if age > 18 and country == "India":
    print("Eligible")
```

They make conditions more expressive and powerful.

13. What happens if we forget a colon (:) after an `if` statement?

Answer:
Python throws a **SyntaxError**, because the colon is required to indicate the start of an indented block.
Example:

```
if x > 5
    print("Hello")
# SyntaxError
```

14. Explain indentation in conditional statements. Why is it important?

Answer:
Python uses indentation instead of braces `{}` to define blocks of code.
Example:

```
if x > 5:
    print("Hello")
```

Incorrect or inconsistent indentation leads to `IndentationError`. It's crucial for readability and structure.

15. Can an `if` condition contain an assignment in Python?

Answer:
No, Python does **not** allow assignments inside `if` conditions (unlike C/C++).
Example:

```
if x = 5:  # invalid in Python
```

Python prevents this to avoid accidental assignments that could cause bugs.

16. What is the difference between `if True:` and `while True:`?

Answer:

- `if True:` executes its block **only once**.
- `while True:` runs in an **infinite loop** until a break condition is met.
  Example:

```
while True:
    break
```

They serve completely different purposes in control flow.

17. How do you handle multiple conditions for the same variable?

Answer:
Use `if-elif-else` chains or Python's chained comparison.
Example:

```
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
else:
    grade = 'C'
```

Or:

```
if 70 <= score < 90:
```

18. What is "pass" statement in conditional blocks?

Answer:
`pass` is a placeholder statement used when a syntactically required block should do nothing.
Example:

```
if x > 10:
    pass  # future code will come here
```

Useful in prototypes or when defining class/function skeletons.

19. How are conditional statements used inside list comprehensions?

Answer:
You can filter or choose values conditionally inside list comprehensions.
Example:

```
result = [x for x in range(10) if x % 2 == 0]
```

Or:

```
values = [x if x > 0 else 0 for x in nums]
```

This helps write compact and efficient code.

20. Why is indentation-based conditional flow considered cleaner than braces?

Answer:
Python's use of indentation:

- improves readability
- eliminates unnecessary symbols
- enforces consistent formatting
  This reduces confusion and makes Python code easy to read compared to languages using `{}`.

Loops

1. What are loops in Python and why are they used?

Answer:
Loops allow a program to execute a block of code repeatedly. Python supports two main loops: `for` and `while`. They are used for tasks like iterating over sequences, processing lists, repeating actions, automation, and pattern generation. Example:

```
for i in range(5):
    print(i)
```

Without loops, repetitive tasks would need manual repetition, making code lengthy and inefficient.

2. What is the difference between `for` loop and `while` loop?

Answer:

- A **for loop** iterates over a sequence (list, tuple, string, range). It is typically used when the number of iterations is known.
- A **while loop** runs until a condition becomes False. It is used when the number of iterations depends on a condition.
- Example:

```
for i in range(3):    # fixed iterations
    print(i)

while x < 5:          # runs until condition ends
    x += 1
```

3. How does a `for` loop work in Python? Explain with an example.

Answer:
Python's `for` loop iterates directly over elements of a sequence, not index-based like C.
Example:

```
for fruit in ["apple", "banana", "mango"]:
    print(fruit)
```

Python internally calls the iterator of the sequence to fetch each item one by one.

4. What is `range()`? How is it used in loops?

Answer:
`range()` generates a sequence of numbers and is commonly used in loops for counting.
Syntax:

```
range(start, stop, step)
```

Example:

```
for i in range(1, 10, 2):
    print(i)   # prints odd numbers
```

It does not create a list in memory; instead, it produces numbers lazily using a range object.

5. What is an infinite loop? How do you prevent it?

Answer:
An infinite loop runs endlessly because the terminating condition never becomes False.
Example:

```
while True:
    print("Hello")
```

Prevent by ensuring:

- conditions eventually become False
- appropriate `break` statements
- counters increment properly
  Infinite loops can freeze programs if not controlled.

6. Explain the use of `break` statement in loops.

Answer:
`break` immediately exits the loop, regardless of the loop's condition.
Example:

```
for i in range(10):
    if i == 5:
        break
```

It is used when the desired result is found or further iteration becomes unnecessary.

7. What is the `continue` statement and how does it work?

Answer:
`continue` skips the current iteration and moves to the next loop cycle.
Example:

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

It helps skip unwanted values without stopping the entire loop.

8. Explain the `pass` statement inside a loop.

Answer:
`pass` is a placeholder that does nothing. It is useful when syntactically a block is required but you do not want code to run yet. Example:

```
for i in range(5):
    pass  # to be implemented later
```

9. What is the `else` block in loops?

Answer:
Python allows an `else` clause after loops.

- In a **for/while loop**, the `else` block runs only if the loop completes normally (not interrupted by `break`).
  Example:

```
for i in range(5):
    print(i)
else:
    print("Loop finished")
```

Useful in search algorithms where `break` indicates early exit.

10. How do nested loops work in Python?

Answer:
A loop inside another loop is called a nested loop.
Example:

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Outer loop runs first; for each iteration, the inner loop executes fully. Nested loops help generate matrices, patterns, and process 2D data.

11. What is the time complexity impact of nested loops?

Answer:
Each nested loop multiplies iterations, increasing time complexity.
Example:

```
for i in range(n):
    for j in range(n):
```

Time complexity becomes **$O(n^2)$**.
More nested layers increase the complexity exponentially, impacting performance.

12. Can we loop through a dictionary? How?

Answer:
Yes, dictionaries support iteration over keys, values, or key-value pairs:

```
for k in dict_obj:
    print(k)
```

```
for v in dict_obj.values():
    print(v)

for k, v in dict_obj.items():
    print(k, v)
```

This is helpful for JSON processing and data manipulation.

13. How do you iterate over a string in Python?

Answer:
A string is an iterable, so you can loop over each character:

```
for ch in "Python":
    print(ch)
```

This is used in text processing and pattern checking.

14. What is list comprehension and how is it related to loops?

Answer:
List comprehension is a concise way to create lists using loops in a single line.
Example:

```
squares = [x*x for x in range(5)]
```

It makes code shorter, faster, and more readable, replacing many traditional loops.

15. How do loops work with the `enumerate()` function?

Answer:
`enumerate()` adds an index automatically when looping through iterable items.
Example:

```
for index, value in enumerate(["a", "b", "c"]):
    print(index, value)
```

Useful when both index and element are needed.

16. How do you iterate through two lists together?

Answer:
Use the `zip()` function.
Example:

```
for name, age in zip(names, ages):
    print(name, age)
```

`zip()` pairs elements from each list, making parallel iteration simple.

17. What is the purpose of `while-else`?

Answer:
The `else` part runs when the `while` loop ends normally (without break).
Example:

```
i = 0
while i < 3:
    i += 1
else:
    print("Completed")
```

Useful in loops where early exit indicates failure or a special condition.

18. Can a `for` loop be converted to a `while` loop? Explain.

Answer:
Yes, any `for` loop can be rewritten using a `while` loop by manually controlling the counter.
Example:

```
i = 0
while i < len(items):
    print(items[i])
    i += 1
```

However, `for` loops are preferred when iterating directly over sequences.

19. How does Python handle loops internally?

Answer:
Python loops rely on the **iterator protocol**, which uses two functions:

- `iter()` → returns an iterator
- `next()` → returns next element
  Example:

```
it = iter([1,2,3])
next(it)
```

The `for` loop internally calls `next()` repeatedly until a `StopIteration` exception occurs.

20. Why does Python encourage `for` loops over `while` loops when possible?

Answer:
Reasons include:

- safer and less error-prone
- no need to manually increment counters
- easier to read and maintain
- prevents infinite loops `for` loops work directly with iterables, making Python more expressive and cleaner.

Python Data Structures

LISTS

1. What is a list in Python?

Answer:
A list is an **ordered, mutable (changeable)** collection of items. It allows storing mixed data types (int, str, float, etc.).
Example:

```
my_list = [10, "Jane", 3.5]
```

Lists are the most commonly used data structure in Python because they support dynamic resizing and multiple operations.

2. How do you create a list?

Answer:
Lists are created using square brackets or the `list()` constructor.
Example:

```
L = [1, 2, 3]
L2 = list((4, 5, 6))
```

3. What are the key features of a Python list?

Answer:

- **Ordered** → items maintain insertion order
- **Mutable** → can be modified (add/remove/update)
- **Heterogeneous** → supports mixed data types
- **Dynamic** → size grows/shrinks automatically
- Supports indexing, slicing, nesting, comprehensions

4. What are common list methods?

Answer:

- `append()` – add item at end
- `insert()` – add at specific index
- `extend()` – add multiple items
- `remove()` – remove first occurrence
- `pop()` – remove item by index
- `sort()` – sort list
- `reverse()` – reverse order

Example:

```
L.append(10)
L.sort()
```

5. What is the difference between list `append()` and `extend()`?

Answer:

- `append()` adds the **entire object** as one element
- `extend()` adds elements **individually**

Example:

```
L = [1, 2]
L.append([3, 4])   # [1, 2, [3, 4]]
L.extend([3, 4])   # [1, 2, 3, 4]
```

6. How does list slicing work?

Answer:
Slicing extracts a portion of the list:

```
L[start:end:step]
```

Example:

```
L[1:4]     # elements from index 1 to 3
L[::-1]    # reverse list
```

7. What is list comprehension?

Answer:
A concise way to create lists.
Example:

```
squares = [x*x for x in range(5)]
```

It is faster and more readable compared to loops.

TUPLES

1. What is a tuple in Python?

Answer:
A tuple is an **ordered, immutable** collection of items.
Example:

```
t = (1, 2, "Mohan")
```

Once created, values cannot be changed.

2. Why are tuples immutable?

Answer:
Immutability:

- provides data integrity

- makes tuples hashable (usable as dictionary keys)
- improves performance (tuples are faster than lists)

3. How do you create a tuple?

Answer:

```
t = (1, 2, 3)
t2 = tuple([4, 5])
```

Special case (single-element tuple):

```
t = (10,)   # comma required
```

4. When should you use a tuple instead of a list?

Answer:

- when data should not change
- for fixed collections (coordinates, database rows)
- when performance is important
- when using data as a dictionary key

5. Can a tuple contain mutable elements?

Answer:
Yes, a tuple can contain lists or other mutable objects.
Example:

```
t = (1, [2, 3])
```

Tuple identity is immutable, but internal mutable items can change.

SETS

1. What is a set in Python?

Answer:
A set is an **unordered collection of unique elements**.
Example:

```
s = {1, 2, 2, 3}   # becomes {1, 2, 3}
```

Duplicates are removed automatically.

2. What are key features of sets?

Answer:

- **Unordered**
- **Unique elements**
- **Mutable**

- **Faster lookup** due to hashing
- Supports set operations (union, intersection, difference)

3. How do you create a set?

Answer:

```
s = {1, 2, 3}
s2 = set([4, 5, 6])
```

Empty set must be created using:

```
s = set()
```

Because `{}` creates an empty dictionary.

4. What are common set operations?

Answer:

- `union()` → combines items
- `intersection()` → common items
- `difference()` → removes items present in another set
- `add()`
- `remove()`

Example:

```
a = {1, 2, 3}
b = {3, 4}

a.union(b)          # {1,2,3,4}
a.intersection(b)   # {3}
```

5. Can sets store mutable elements?

Answer:
No.
Sets can only store **hashable** (immutable) items like numbers, strings, tuples.
Lists or dictionaries cannot be elements of a set.

STRINGS

1. What is a string in Python?

Answer:
A string is an **immutable sequence of characters** enclosed in quotes.
Example:

```
text = "Hello"
```

2. What makes strings immutable?

Answer:
Once created, string contents cannot be changed.
Example:

```
s = "Python"
s[0] = "J"   # Error
```

Instead, new strings are created in memory.

3. What are common string operations?

Answer:

- indexing → `s[0]`
- slicing → `s[1:4]`
- concatenation → `"A" + "B"`
- repetition → `"Hi"*3`
- membership → `"a" in "apple"`
- formatting → f-strings

Methods:

- `upper()`
- `lower()`
- `split()`
- `replace()`
- `strip()`

Example:

```
"hello".upper()
```

4. How does string slicing work?

Answer:

```
s[start:end:step]
```

Example:

```
s = "Python"
s[1:4]   # "yth"
s[::-1]  # reverse string
```

5. Why are strings hashable in Python?

Answer:
Because they are immutable, their hash value never changes.
Thus, strings can be used as dictionary keys.

DICTIONARIES

1. What is a dictionary in Python?

Answer:
A dictionary is an **unordered collection of key-value pairs**.
Example:

```
student = {"name": "Hitesh", "age": 21}
```

Keys must be unique and immutable; values can be any type.

2. How do you create a dictionary?

Answer:

```
d = {"a": 1, "b": 2}
d2 = dict(name="Himanshu", age=22)
```

3. What are dictionary keys and values?

Answer:

- **Key** → must be unique & immutable (str, int, tuple)
- **Value** → can be any Python object

You access values by key:

```
student["name"]
```

4. What are common dictionary methods?

Answer:

- `keys()`
- `values()`
- `items()`
- `get()`
- `update()`
- `pop()`
- `popitem()`

Example:

```
student.get("age")
student.update({"city": "Indore"})
```

5. What is the difference between `get()` and using brackets for key access?

Answer:

- `d[key]` → raises error if key not found
- `d.get(key)` → returns `None` or default value

Example:

```
student.get("salary", 0)
```

Functions

1. What is a function in Python? Why are functions used?

Answer:
A function is a block of reusable code used to perform a specific task. Functions help break a program into smaller, manageable parts and promote code reusability, modularity, readability, and faster development.
Example:

```
def greet():
    print("Hello")
```

Functions prevent repetition of code and make programs cleaner and more efficient.

2. How do you define and call a function in Python?

Answer:
A function is defined using the `def` keyword followed by the function name and parentheses.
Example:

```
def add(a, b):
    return a + b

add(5, 3)   # calling function
```

Indentation defines the function block, and `return` sends a value back to the caller.

3. What is the difference between a function and a method?

Answer:

- A **function** is an independent block of code that is not tied to an object.
- A **method** belongs to an object and is defined inside a class.
  Example:

```
len("Hello")          # built-in function
"Hello".upper()       # method of string object
```

Methods implicitly receive the object as the first argument (`self` in classes).

4. What are the different types of function arguments in Python?

Answer:
Python supports several types of arguments:

1. **Positional arguments**
2. **Keyword arguments**
3. **Default arguments**
4. **Variable-length arguments (args)*
5. **Keyword variable-length arguments **(kwargs)**

Example:

```
def example(a, b=2, *args, **kwargs):
    pass
```

This flexibility makes Python functions powerful and easy to work with.

5. What are default arguments? Why are they useful?

Answer:
Default arguments provide a predefined value if no value is passed by the caller.
Example:

```
def greet(name="User"):
    print("Hello", name)
```

They reduce code redundancy and allow optional parameters.

6. What is the difference between *args and **kwargs?

Answer:

- `*args` allows passing **variable number of positional arguments**.
- `**kwargs` allows passing **variable number of keyword arguments**.
  Example:

```
def func(*args, **kwargs):
    print(args)
    print(kwargs)
```

These are used when the number of inputs is unknown in advance.

7. What is the return statement? Can a function return multiple values?

Answer:
The `return` statement sends a value back to the caller and terminates the function.
Python can return multiple values using tuples:
Example:

```
def calc(a, b):
    return a+b, a-b

sum_, diff = calc(10, 5)
```

This feature makes Python functions versatile.

8. What is a lambda function? How is it different from a normal function?

Answer:
A lambda function is an anonymous, one-line function defined using the `lambda` keyword.
Example:

```
square = lambda x: x*x
```

Differences:

- No name
- Single expression only
- Cannot contain statements
  Useful for short, simple operations like sorting or filtering.

9. What is a docstring? Why is it important?

Answer:
A docstring is a documentation string written inside a function to describe its purpose.
Example:

```
def add(a, b):
    """This function returns the sum of two numbers."""
    return a + b
```

Docstrings improve readability, help generate documentation, and can be accessed using `help()` or `.__doc__`.

10. Explain variable scope in Python (local vs global).

Answer:

- **Local variables** exist only within a function.
- **Global variables** exist throughout the program.
  Example:

```
x = 10  # global
def func():
    y = 5  # local
```

Local variables override global ones if names match. Python resolves scope using **LEGB rule** (Local → Enclosing → Global → Built-in).

11. What is the `global` keyword? When do we use it?

Answer:
The `global` keyword allows modifying a global variable inside a function.
Example:

```
x = 10
def modify():
    global x
    x = 20
```

Without `global`, assignments inside a function create a new local variable.

12. What is the `nonlocal` keyword?

Answer:
`nonlocal` is used inside nested functions to modify a variable from the outer (enclosing) function.
Example:

```
def outer():
    x = "outer"
    def inner():
        nonlocal x
        x = "modified"
```

It bridges the scope between local and global.

13. What is recursion? Give an example.

Answer:
Recursion is when a function calls itself to solve smaller instances of a problem.
Example:

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

Recursion is useful for tree traversal, factorial, Fibonacci, etc., but may cause stack overflow if not used carefully.

14. What are built-in functions in Python? Give examples.

Answer:
Python provides several built-in functions for common tasks. Examples:
`len()`, `sum()`, `max()`, `min()`, `range()`, `type()`, `id()`, `sorted()`
Example:

```
len([1,2,3])   # 3
```

Built-in functions improve productivity and reduce code complexity.

15. What are higher-order functions?

Answer:
Higher-order functions can take other functions as arguments or return them.
Examples:

- `map()`
- `filter()`
- `reduce()`
  Example:

```
list(map(lambda x: x*x, [1,2,3]))
```

They support functional programming paradigms.

16. What is function overloading? Does Python support it?

Answer:
Python does not support traditional function overloading like Java or C++.
If you define two functions with the same name, the latest one overwrites the previous one.
However, Python *supports similar behavior* using default arguments or `*args`.

17. What is function hoisting? Does Python allow calling a function before definition?

Answer:
Python **does not** support hoisting.
A function must be defined before it is called, otherwise Python raises a `NameError`.
Example:

```
hello()      # Error
def hello():
    pass
```

18. Can a function be assigned to a variable or passed as an argument?

Answer:
Yes, Python treats functions as first-class citizens.
Example:

```
def hello():
    print("Hi")

x = hello   # assign function
x()         # call using variable
```

This enables functional programming patterns and decorators.

19. What is a generator function? How is it different from a normal function?

Answer:
A generator function uses the `yield` keyword to return values one at a time, pausing after each yield.
Example:

```
def count():
    yield 1
    yield 2
```

Generator vs normal function:

- Generator provides lazy evaluation

- Saves memory
- Used for large data processing

20. What are decorators in Python?

Answer:
Decorators modify or enhance the behavior of another function without changing its code.
Example:

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper
```

They are widely used in logging, authorization, performance monitoring, and frameworks like Flask/Django.

Exception Handling

1. What is an exception in Python? How is it different from an error?

Answer:
An exception is an unexpected event that interrupts the normal flow of a program, such as dividing by zero or accessing invalid indices.
Errors are mostly related to issues in code syntax or logic, whereas exceptions occur during runtime.
Example:

```
x = 10 / 0   # ZeroDivisionError
```

Exception handling allows programs to fail gracefully instead of crashing abruptly.

2. What is exception handling? Why is it important?

Answer:
Exception handling is the mechanism to handle runtime errors safely using `try`, `except`, `else`, and `finally` blocks.
It ensures:

- program stability
- prevention of abrupt termination
- meaningful error messages
- controlled recovery
  Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

3. What are the main keywords used for exception handling in Python?

Answer:
Python uses the following keywords:

- **try** → wrap risky code
- **except** → handle exceptions
- **else** → runs if no exception occurs
- **finally** → always executes (cleanup code)
- **raise** → manually raise exceptions
- **assert** → used for debugging checks
  These help make the program more robust and reliable.

4. How does a try–except block work? Explain with an example.

Answer:
The code in the `try` block executes first. If an exception occurs, control shifts to the `except`

block.
Example:

```
try:
    num = int("abc")
except ValueError:
    print("Invalid conversion")
```

If no exception occurs, `except` block is skipped.

5. What is the use of the else block in exception handling?

Answer:
The `else` block runs only if the `try` block does **not** raise any exception.
Example:

```
try:
    x = int("10")
except:
    print("Error")
else:
    print("Conversion successful")
```

`else` is helpful when you want to separate error-handling code from success code.

6. What is the purpose of the finally block?

Answer:
`finally` executes **always**, regardless of whether an exception occurs.
Common uses:

- closing files
- releasing resources
- closing database connections
  Example:

```
try:
    file = open("data.txt")
finally:
    file.close()
```

7. Can a try block exist without except?

Answer:
Yes, a `try` block must have **either**:

- an `except` block
  OR
- a `finally` block
  Example:

```
try:
    risky()
```

```
finally:
    cleanup()
```

But a `try` block cannot exist alone.

8. What is multiple except handling? Why is it useful?

Answer:
Python allows multiple `except` blocks to handle different error types separately.
Example:

```
try:
    x = int(value)
except ValueError:
    print("Not a number")
except TypeError:
    print("Invalid type")
```

It provides more specific and meaningful error messages.

9. How do you catch multiple exceptions in a single except block?

Answer:
Use a tuple of exception types.
Example:

```
try:
    result = 10 / value
except (ZeroDivisionError, TypeError):
    print("Error occurred")
```

Useful when the handling logic is the same for different exceptions.

10. What is the difference between Exception and BaseException?

Answer:

- `BaseException` is the root of all exceptions, including system-exiting exceptions like `SystemExit`, `KeyboardInterrupt`.
- `Exception` is the base class for most common runtime errors.
  In practice, we catch `Exception` to avoid intercepting exit or interrupt events.

11. How do you manually raise an exception in Python?

Answer:
Use the `raise` keyword to generate an exception intentionally.
Example:

```
if age < 18:
    raise ValueError("Age must be 18+")
```

It is useful for validating inputs or enforcing business rules.

12. What is the assert statement? How does it work?

Answer:
`assert` is used for debugging and validating conditions during development.
Example:

```
assert x > 0, "x must be positive"
```

If the condition is False, AssertionError is raised. In production, assertions can be disabled for performance.

13. What is a custom exception? Why do we create one?

Answer:
Custom exceptions allow developers to define application-specific error types.
Example:

```
class InvalidAgeError(Exception):
    pass
```

Custom exceptions improve readability and allow more precise error handling.

14. Explain how to create and use a custom exception.

Answer:
Step 1 — Define the exception class by inheriting from `Exception`:

```
class NegativeNumberError(Exception):
    pass
```

Step 2 — Raise it when needed:

```
if num < 0:
    raise NegativeNumberError("Negative value not allowed")
```

Step 3 — Handle it:

```
try:
    check()
except NegativeNumberError as e:
    print(e)
```

15. Can we write nested try–except blocks?

Answer:
Yes, nested try blocks allow handling errors at different levels.
Example:

```
try:
    try:
```

```
        x = int("abc")
    except ValueError:
        print("Inner exception")
except Exception:
    print("Outer exception")
```

Useful in complex applications like file operations, database transactions, etc.

16. What happens if an exception is not handled?

Answer:
The program stops executing and displays a traceback message.
Example:

```
ZeroDivisionError: division by zero
```

Unhandled exceptions lead to program failure, which is why handling them is important.

17. How do exception hierarchies work in Python?

Answer:
Exceptions follow an inheritance tree.
Example:
`IndexError` → subclass of `LookupError` → subclass of `Exception`
Catching a parent exception will also catch its child exceptions.
Example:

```
except LookupError:
```

This will catch both `IndexError` and `KeyError`.

18. What is the best practice when catching exceptions?

Answer:

- Catch specific exceptions, not generic `Exception`
- Avoid empty `except` blocks
- Use finally for cleanup
- Log exceptions for debugging
  Good practice example:

```
try:
    process()
except FileNotFoundError:
    print("File missing")
```

19. Why is a bare except (without exception type) discouraged?

Answer:
A bare `except:` catches all kinds of exceptions, including system-level ones like `KeyboardInterrupt`, which can make debugging harder.

It hides real problems and may cause infinite loops or silent failures.
Only use it when absolutely necessary.

20. How does Python internally handle exceptions?

Answer:
When an exception occurs:

1. Python creates an exception object
2. It searches for the nearest matching `except` block
3. If found → handles it
4. If not found → terminates the program
   The process is similar to throwing and catching exceptions in other languages.

File Handling

1. What is file handling in Python? Why is it important?

Answer:
File handling refers to reading from and writing to files stored on the computer's filesystem using Python.
It is important for tasks such as:

- reading configuration files
- storing logs
- data storage & processing
- handling CSV/JSON files
- saving program outputs
  Python provides built-in functions like `open()`, `read()`, `write()`, and supports multiple file formats.

2. How do you open a file in Python? Explain the syntax.

Answer:
Python uses the `open()` function to open files.
Syntax:

```
open(filename, mode)
```

Example:

```
file = open("data.txt", "r")
```

`mode` specifies whether the file is opened for reading, writing, or appending.
Always close the file using `file.close()` or use the `with` statement for automatic closure.

3. What are the different file modes available in Python?

Answer:
Common file modes include:

- `"r"` – read (default)
- `"w"` – write (overwrites file)
- `"a"` – append
- `"x"` – create file (error if exists)
- `"b"` – binary mode
- `"t"` – text mode (default)
- `"r+"` – read and write
  Example:

```
open("img.png", "rb")  # binary read
```

Modes let you control how the file is accessed.

4. What is the difference between `read()`, `readline()`, and `readlines()`?

Answer:

- `read()` → reads the entire file as a single string
- `readline()` → reads one line at a time
- `readlines()` → returns a list of all lines

Example:

```
f.read()        # "full file"
f.readline()    # "first line"
f.readlines()   # ["line1", "line2", ...]
```

Choose based on file size and memory usage.

5. Why should we use the `with` statement when working with files?

Answer:
The `with` statement automatically opens and closes the file, even if errors occur.
Example:

```
with open("data.txt", "r") as f:
    content = f.read()
```

Benefits:

- cleaner syntax
- no need for `close()`
- prevents file locks and resource leaks
  It is the recommended way to handle files in Python.

6. How do you write data to a file in Python?

Answer:
Use `write()` or `writelines()` methods.
Example:

```
with open("output.txt", "w") as f:
    f.write("Hello World")
```

If the file exists, `"w"` mode overwrites it; `"a"` mode adds new content at the end.

7. How do you append data to an existing file?

Answer:
Use the `"a"` (append) mode.
Example:

```
with open("log.txt", "a") as f:
```

```
    f.write("\nNew log entry")
```

This adds content without deleting existing data.

8. How do you check if a file exists in Python?

Answer:
Using the `os.path` module:

```
import os
os.path.exists("file.txt")
```

Or using `Path` from `pathlib`:

```
from pathlib import Path
Path("file.txt").exists()
```

Useful for avoiding errors like `FileNotFoundError`.

9. What is the difference between text mode and binary mode?

Answer:

- **Text mode** (`"t"`) reads data as string; handles encoding automatically.
- **Binary mode** (`"b"`) reads data as bytes; used for images, audio, PDF, etc.
  Example:

```
open("image.jpg", "rb")
```

Binary mode preserves raw file content.

10. How does Python handle file encoding?

Answer:
By default, Python uses system encoding (usually UTF-8).
You can specify encoding explicitly:

```
open("file.txt", "r", encoding="utf-8")
```

This prevents issues with special characters, especially when transferring files across systems.

11. What happens when you open a file in write mode (`"w"`) that already exists?

Answer:
The file is **overwritten**, and its previous content is deleted.
Example:

```
open("data.txt", "w")
```

Use `"a"` or `"x"` if you want to preserve or protect existing content.

12. How do you delete a file in Python?

Answer:
Using the `os` module:

```
import os
os.remove("data.txt")
```

Or using `pathlib`:

```
from pathlib import Path
Path("data.txt").unlink()
```

Useful for managing temporary or outdated files.

13. What is `tell()` and `seek()` used for in file handling?

Answer:

- `tell()` returns the current cursor (file pointer) position in bytes.
- `seek()` moves the cursor to a specific position.
  Example:

```
f.seek(0)     # move to start
pos = f.tell()  # get cursor position
```

These functions help navigate large files efficiently.

14. How do you read a file line by line without loading entire file in memory?

Answer:
Use a loop to iterate over file objects:

```
with open("largefile.txt") as f:
    for line in f:
        print(line)
```

This handles large files efficiently by reading one line at a time.

15. How do you handle file-related exceptions?

Answer:
Use `try-except` blocks to catch errors like missing files or permission issues.
Example:

```
try:
    with open("data.txt") as f:
        content = f.read()
except FileNotFoundError:
```

```
    print("File not found")
```

This ensures graceful handling instead of program crash.

16. What are CSV files? How do you read/write them in Python?

Answer:
CSV (Comma-Separated Values) files store tabular data.
Python supports CSV operations via the `csv` module:
Reading:

```
import csv
with open("data.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Writing:

```
with open("data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Age"])
```

17. How do you read and write JSON files?

Answer:
Python provides the `json` module.
Reading:

```
import json
with open("data.json") as f:
    data = json.load(f)
```

Writing:

```
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

JSON is widely used in APIs and data exchange.

18. What is buffering in file handling?

Answer:
Buffering temporarily stores file data in memory before writing it to disk.
Python allows specifying buffer size:

```
open("file.txt", "w", buffering=1024)
```

Larger buffers improve performance for big write operations.

19. How do you rename or move a file?

Answer:
Using `os` module:

```
import os
os.rename("old.txt", "new.txt")
```

Using `shutil` module:

```
import shutil
shutil.move("file.txt", "folder/")
```

Useful for organizing files.

20. What are binary files? Give examples.

Answer:
Binary files store data in non-text format such as bytes.
Examples:

- images (.jpg, .png)
- audio (.mp3)
- videos (.mp4)
- executable files
  Python reads/writes them using `"rb"` and `"wb"` modes.
  Example:

```
with open("pic.jpg", "rb") as f:
    data = f.read()
```

Python Object-Oriented Programming

1. What is OOP (Object-Oriented Programming)? Why is it used?

Answer:
OOP is a programming paradigm based on the concept of **objects**, which bundle data (attributes) and behavior (methods) together.
Python supports OOP to make programs structured, modular, reusable, and easier to maintain.

Benefits of OOP:

- **Modularity** → code divided into classes
- **Reusability** → through inheritance
- **Flexibility** → through polymorphism
- **Security** → through encapsulation
  OOP helps model real-world entities like Students, Cars, Employees into software objects.

2. What is a class in Python?

Answer:
A class is a **blueprint** or **template** for creating objects. It defines attributes and methods that objects will have.

Example:

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def drive(self):
        print("Driving...")
```

3. What is an object? How is it created?

Answer:
An object is an **instance** of a class. It contains actual data stored in memory.

Example:

```
mycar = Car("BMW")
print(mycar.brand)
```

`mycar` is an object with its own brand attribute.

4. What is the `__init__` method? Why is it important?

Answer:
`__init__` is a **constructor** that initializes object attributes at creation time.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

It automatically runs when an object is created.

5. What is a constructor? Differentiate default and parameterized constructors.

Answer:
A constructor initializes an object's state.

Types:

- **Default constructor:** no parameters
- **Parameterized constructor:** takes arguments

Example:

```
class A:
    def __init__(self, x=0):  # default & parameterized
        self.x = x
```

Python does not allow multiple constructors; we use default values instead.

6. What is a destructor? When is `__del__()` called?

Answer:
A destructor is the method that is called when an object is garbage collected or deleted.
Python destructor:

```
def __del__(self):
    print("Object destroyed")
```

It executes when the last reference to an object is removed.

7. Explain Encapsulation with an example.

Answer:
Encapsulation means **protecting data** by restricting direct access.
Python uses:

- `_variable` → protected
- `__variable` → private

Example:

```
class Bank:
    def __init__(self):
        self.__balance = 1000  # private variable

    def deposit(self, amount):
```

```
        self.__balance += amount
```

Private variables ensure controlled access through methods.

8. What are Access Modifiers in Python?

Answer:
Python supports three types (based on naming conventions):

1. **Public**
2. `name = "Ramesh"`
3. **Protected** (not enforced)
4. `_salary = 50000`
5. **Private** (name mangling `_ClassName__variable`)
6. `__pin = 1234`

Python does not strictly enforce access control but follows convention-based privacy.

9. What is Abstraction? How is it implemented in Python?

Answer:
Abstraction means **hiding unnecessary details** and showing only essential info.

Implemented using:

- Abstract classes (`from abc import ABC, abstractmethod`)
- Interfaces (conceptual; Python uses abstract base classes)

Example:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
```

Such classes cannot be instantiated until all abstract methods are implemented.

10. Explain Inheritance and its types in Python.

Answer:
Inheritance allows one class (child) to acquire properties of another class (parent).

Types of inheritance:

1. **Single Inheritance**
2. `class B(A): pass`
3. **Multilevel Inheritance**
4. `class C(B): pass`
5. **Multiple Inheritance**

6. `class C(A, B): pass`
7. **Hierarchical Inheritance**
8. `class B(A), class C(A)`
9. **Hybrid Inheritance** (combination)

11. What is Polymorphism? Give examples.

Answer:
Polymorphism means **many forms** → same function name behaving differently depending on the object.

Examples:

Method Overriding

Child class redefines method of parent class.

```
class A:
    def show(self): pass
class B(A):
    def show(self): print("B show")
```

Duck Typing

"If it walks like a duck, it's a duck."

```
def func(obj):
    obj.start()
```

Operator Overloading

Using dunder methods.

```
def __add__(self, other)
```

12. What is method overriding? How is it useful?

Answer:
Overriding is redefining a method in a child class that already exists in parent class.
Useful for customizing behavior.

Example:

```
class Animal:
    def sound(self): print("Some sound")
class Dog(Animal):
    def sound(self): print("Bark")
```

13. Does Python support method overloading?

Answer:
Python does **not** support traditional method overloading.
You can achieve similar behavior using:

- default arguments
- `*args`
- conditional logic

Example:

```
def add(a, b=0):
    return a + b
```

14. What is operator overloading? How is it done in Python?

Answer:
Operator overloading allows custom behavior for operators like +, -, *, ==.

Using dunder methods:
Examples:

- `__add__()` for `+`
- `__eq__()` for `==`
- `__mul__()` for `*`

Example:

```
class Point:
    def __init__(self, x):
        self.x = x
    def __add__(self, other):
        return Point(self.x + other.x)
```

15. What is `super()`? Why is it used?

Answer:
`super()` is used to call the parent class constructor or method.

Example:

```
class B(A):
    def __init__(self):
        super().__init__()
```

Useful in:

- inheritance
- multiple inheritance
- maintaining method resolution order (MRO)

16. Explain Method Resolution Order (MRO).

Answer:
MRO defines the order in which Python searches for a method in inheritance hierarchy.

Python uses **C3 Linearization**, viewable using:

```
ClassName.mro()
```

In multiple inheritance, MRO avoids conflicts and ensures consistency.

17. What are class variables vs instance variables?

Answer:

Instance variables

Unique to each object.
Set inside `__init__`.

Class variables

Shared by all objects of the class.
Defined outside methods.

Example:

```
class A:
    species = "Human"   # class variable
    def __init__(self, name):
        self.name = name   # instance variable
```

18. What are static methods? How are they different from class methods?

Static Method

- Defined with `@staticmethod`
- Does not use `self` or `cls`
- Behaves like normal functions inside class

Class Method

- Defined with `@classmethod`
- First argument is `cls`
- Used to modify class-level attributes

Example:

```
class A:
    @staticmethod
    def greet(): pass

    @classmethod
    def update(cls): pass
```

19. What is `self` in Python classes?

Answer:
`self` refers to the **current object instance**.
It is used to access attributes/methods of the object.

Example:

```
def setname(self, name):
    self.name = name
```

Python automatically passes the object as the first argument.

20. What are dunder (magic) methods? Give examples.

Answer:
Dunder methods start and end with double underscores (`__method__`).
Examples:

- `__init__` – constructor
- `__str__` – string representation
- `__len__` – len(obj)
- `__add__` – operator overloading
- `__del__` – destructor

They customize built-in behavior and make classes behave like built-in types.

21. What are Getter and Setter methods?

Answer:
They control access to private variables.
Python uses `@property` decorator for getters.

Example:

```
class A:
    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, value):
        self.__age = value
```

This is a better way to handle data validation.

22. What is Composition in OOP? How is it different from Inheritance?

Answer:
Composition means building complex objects by **combining other objects**.

Example:

```
class Engine:
    pass

class Car:
    def __init__(self):
```

```
        self.engine = Engine()
```

Difference:

- Inheritance: IS-A relationship
- Composition: HAS-A relationship

Composition is preferred when the relationship is not permanent.

23. What is Aggregation?

Answer:
Aggregation is a weaker form of composition where the child object can exist independently of the parent.

Example:

```
class Classroom:
    def __init__(self, students):
        self.students = students
```

Students exist even without the classroom.

24. What is multiple inheritance? What problems can it cause?

Answer:
Multiple inheritance means a class inherits from more than one parent.

Example:

```
class C(A, B):
    pass
```

Problem:

- Method conflicts
- Complex MRO
  Python solves this through **C3 linearization**.

25. Explain "Duck Typing" in Python OOP.

Answer:
Duck Typing means object type is less important than the method it defines.

If an object behaves like a duck, Python treats it as a duck.

Example:

```
class A:
    def start(self): pass
```

```
def func(obj):
    obj.start()
```

Python doesn't care about the class, only the method presence.

26. What is an interface? Does Python support interfaces?

Answer:
Python doesn't have official interfaces like Java, but we can create them using **Abstract Base Classes** (ABC).

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def draw(self):
        pass
```

27. What is the difference between abstraction and encapsulation?

Answer:

- **Abstraction** → Hides complexity
- **Encapsulation** → Protects data
  Example:
  ATM →
  User sees buttons (abstraction), but internal logic is hidden and protected (encapsulation).

28. Can you create private methods in Python?

Answer:
Yes → prefix with __.

Example:

```
def __secret(self):
    pass
```

Python applies **name mangling** → `_ClassName__secret`.

29. What is object cloning in Python?

Answer:
Using the `copy` module:

- `copy.copy()` → shallow copy
- `copy.deepcopy()` → deep copy

Example:

```
import copy
b = copy.deepcopy(a)
```

30. Why is Python considered more flexible in OOP than other languages?

Answer:
Because Python supports:

- dynamic typing
- duck typing
- multiple inheritance
- operator overloading
- mixin classes
- flexible access modifiers

Python OOP is powerful yet easy to use.